C as Portable Assembly: From Hardware Diversity to a Minimal Machine and Canonical Forms

Made by AI

***Abstract*—Programmers are often taught computer hardware and the C language as collections of categories to memorize: CISC and RISC, L1, L2, and L3 caches, arrays, pointers, prefix and postfix increment. This tutorial argues that most of these categories can instead be derived from a small number of ideas. We first show how processor architectures diverged, so that the same operation is performed by different instructions, of different sizes and costs, on different machines, and explain the categories that engineers introduced to describe that divergence. We then identify a minimal set of practical operations, such as load, store, add, compare, and branch, on which nearly every general-purpose machine agrees, and explain why C, though not originally designed with portability as its main goal, came to serve as a "portable assembly" over that common core. Finally, we argue that C's portability rests on a precise bargain: the language defines behavior only where hardware agrees and deliberately leaves gaps where it does not. We also show that the hardware knows addresses but not lifetimes, explain how C's static, automatic, and manually allocated storage are built from ordinary operations, and examine where the abstraction must meet the real machine: device registers and `volatile`, calling conventions, and concurrency. Code written in canonical forms that stay inside the agreed region translates correctly on every machine; code that steps into the gaps does not. Short compiler listings for x86-64, AArch64, 32-bit RISC-V, and Arm Cortex-M0 illustrate each step.**

***Index Terms*—Abstract machine, application binary interface, C language, canonical forms, concurrency, memory management, instruction set architecture, portability, undefined behavior.**

I. Introduction

A student learning systems programming meets a long list of names. Processors are CISC or RISC. Memory comes in L1, L2, and L3 caches. C has arrays and pointers, structs and unions, and three ways to add one to a variable (`i = i + 1`, `++i`, and `i++`), along with advice about which is faster. It is easy to come away believing that expertise consists of remembering all of these categories and the rules attached to them.

This paper takes the opposite view. Most of these categories are names for consequences of a few underlying facts, and once the facts are understood the categories can be derived rather than memorized. The approach follows the spirit of Drepper's study of memory [1], which explained cache behavior from the structure of the hardware, but applies it to the relationship between hardware and the C language.

The argument is built in layers, illustrated in Fig. 1. Section II shows how hardware diverged, so that one operation costs different amounts on different machines, and Section III explains why engineers introduced categories to describe the divergence. Section IV identifies a *minimal machine*: a small set of practical operations on which nearly every general-purpose processor agrees. Section V explains why C came to serve as portable assembly over that common core, and how it keeps the promise by leaving deliberate gaps where hardware disagrees. Section VI explains how C manages memory, both automatically and by hand, using nothing but those operations, and Section VII examines the three places where the abstraction must meet the real machine: hardware devices, binary interfaces, and concurrent processors. Section VIII shows why, as a consequence, each C construct needs a *canonical form*: code written outside those forms falls into the gaps, and the abstraction no longer translates correctly. Section IX discusses limits, and Section X concludes.

The evidence offered is deliberately light. Where a claim concerns what compilers do, we show short listings produced by Clang at optimization level `-O2` for four targets: x86-64, the 64-bit Arm architecture (AArch64), the 32-bit RISC-V base integer set (RV32I), and the Arm Cortex-M0 microcontroller. The listings illustrate the argument; they are not benchmarks.

II. How Hardware Diverged

Every stored-program computer descends from the design described by von Neumann in 1945, in which instructions and data share one memory and a processor fetches and executes instructions in sequence [2]. Beyond that shared idea, processor designers have made very different choices about what a single instruction should do and how it should be encoded. The result is that the same operation is carried out in different ways, at different costs, on different machines.

A. One Operation, Several Encodings

Consider the simplest possible piece of work: adding one to an integer stored in memory, written in C as `*p += 1`. Table I shows what four targets produce.

Manuscript prepared November 2026.

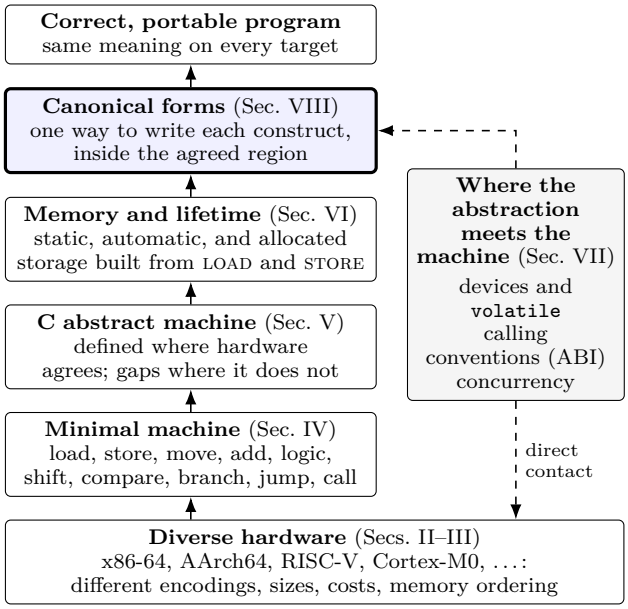


Fig. 1. The structure of the argument. The main column shows each layer abstracting the one below it. The side box marks the places where a program must reach past the abstraction to the hardware itself. Canonical forms, including those for hardware access, are what make the whole stack hold on every target.

TABLE I
MACHINE CODE FOR `*p += 1` ON FOUR TARGETS (CLANG `-O2`)

Target	Instructions	Count	Bytes
x86-64	`incl (%rdi)`	1	2
AArch64	`ldr; add; str`	3	12
RISC-V RV32I	`lw; addi; sw`	3	12
RISC-V RV32IC	`lw; addi; sw` (compressed)	3	6
Cortex-M0 (Thumb)	`ldr; adds; str`	3	6

The x86-64 processor performs the whole operation with one instruction that reads memory, adds, and writes memory back, encoded in two bytes. The other three use three separate instructions: *load* the value into a register, *add* one, and *store* it back. AArch64 and the RISC-V base set encode every instruction in four bytes, so the same work takes twelve bytes. The RISC-V compressed extension and Arm's Thumb encoding use two-byte forms of the same three instructions, halving the size again. Five correct programs, three different instruction counts, three different sizes: and none of these numbers says, by itself, which runs fastest, because that depends on how each processor executes its instructions internally.

B. Missing Operations

Machines also differ in which operations they provide at all. Table II shows integer multiplication and division.

The RISC-V base integer set has no multiply or divide instructions; they belong to an optional extension [3]. The Cortex-M0 can multiply but cannot divide in hardware [4]. On those machines the compiler calls a library routine that performs the operation in software, using the instructions the machine does have [5]; on the Cortex-M0 the routine is the one defined by Arm's run-time ABI. This answers a natural worry: a correct program does not fail on a machine that lacks an instruction it needs. It still runs, but the missing operation is built from simpler ones, at a higher cost.

TABLE II
INTEGER MULTIPLY AND DIVIDE ON FOUR TARGETS (CLANG `-O2`)

Target	`a * b`	`a / b`
x86-64	`imull`	`idivl`
AArch64	`mul`	`sdiv`
RISC-V RV32I	call `__mulsi3`	call `__divsi3`
Cortex-M0	`muls`	call `__aeabi_idiv`

C. Width, Order, and Alignment

Divergence extends to the shape of data. Processors differ in the natural width of their registers and of an addition (16, 32, or 64 bits); in the order in which the bytes of a multi-byte value are stored in memory, known as byte order or endianness; and in whether a value must be stored at an address that is a multiple of its size, known as alignment. They also differ in instruction length: an x86 instruction may be anywhere from one to fifteen bytes long [6], whereas the RISC-V base set uses a fixed four bytes [3], as does AArch64 (Table I).

D. Size Is Not Speed

It would be natural to assume that fewer instructions, or fewer bytes, mean faster execution. They do not, for three reasons. First, instructions differ greatly in *latency*, the time before their result is available. On Intel's Skylake processors, for example, an integer addition has a latency of one cycle and a multiplication three, while a 32-bit division takes 26 [7]; a division that one machine performs with a single instruction may therefore cost far more than many additions on another. Second, modern processors overlap the execution of many instructions at once, and many decompose complex instructions into simpler internal steps, so the instruction count visible in a listing is only a rough guide to the work performed [8]. Third, and most important, an instruction that touches memory takes as long as the memory takes to answer, and that depends on where the data is (Section III). The cost of an operation is thus a property of the operation, the machine, and the data together; no listing shows it by itself.

Key point. The same operation is not one thing in hardware. It is a family of instruction sequences whose length, size, and speed depend on the machine.

III. Why Engineers Introduced Categories

Faced with this variety, engineers introduced names to describe the important differences. The names are useful, but they are best understood as summaries of underlying facts, not as facts in their own right.

A. Architecture and Implementation

The most important distinction dates from IBM's System/360 of 1964, whose designers separated a machine's *architecture*, the behavior visible to a programmer, from its *implementation*, the hardware that delivers that behavior [9]. One architecture could then be implemented by a family of machines of very different speed and price, all running the same programs. Today the architecture is called the *instruction set architecture* (ISA) and the implementation the *microarchitecture*. This one distinction explains much of what follows: anything in the ISA is a promise that programs can rely on; anything in the microarchitecture can change from one chip to the next.

B. CISC and RISC

The labels CISC (complex instruction set computer) and RISC (reduced instruction set computer) describe where a design places its complexity. Early designs, like x86, offered powerful instructions that combine several steps, such as the memory increment of Table I, in variable-length encodings. In 1980 Patterson and Ditzel argued that ever more complex instruction sets were not cost-effective, and that a simpler instruction set, suited to single-chip implementation and to compilers, would do better overall [10]. The designs that followed used instructions of fixed length and touched memory only through load and store. Seen this way, the two labels are not two kinds of machine to memorize, but two answers to one question: should the processor or the compiler assemble simple steps into larger ones? Modern processors blur the line, and many CISC processors internally split complex instructions into simpler steps [8].

C. The Memory Hierarchy

The names L1, L2, and L3 arose from a different problem. Processor speed grew much faster than the speed of main memory, a widening gap that, Wulf and McKee warned, would eventually make systems hit a *memory wall*, with performance limited by memory alone [11]. The remedy, introduced in some of the first commercial machines with one, such as the IBM System/360 Model 85 of 1968 [12], is a small, fast memory, the *cache*, that keeps recently used data close to the processor. As the gap widened, designers added more caches, each larger and slower than the last. The level numbers record nothing more than distance from the processor. They need not be memorized as separate concepts: a single principle, that data farther from the processor takes longer to reach and moves in fixed-size blocks, explains their behavior [1], [8]. Caches belong to the microarchitecture, which is why the ISA, and therefore C, does not mention them.

D. Byte Order and Data Models

Two further sets of names describe the shape of data, and both matter directly to C. The terms *big-endian* (most significant byte first) and *little-endian* (least significant byte first) were borrowed by Cohen in 1981 from the egg-breaking factions of *Gulliver's Travels* [13], a reminder that the choice is a convention rather than a law of nature: x86 is little-endian, many network protocols are big-endian, and some architectures can operate either way.

TABLE III
Type Sizes in Bytes by Target (Clang)

Target	`int`	`long`	`long long`	pointer	Model
x86 (32-bit) Linux	4	4	8	4	ILP32
RISC-V RV32	4	4	8	4	ILP32
x86-64 Linux	4	8	8	8	LP64
AArch64 Linux	4	8	8	8	LP64
RISC-V RV64	4	8	8	8	LP64
x86-64 Windows	4	4	8	8	LLP64
AArch64 Windows	4	4	8	8	LLP64

The *data model* records how wide C's basic types are on a given platform. Its names abbreviate which types are 64 bits wide: ILP32 means `int`, `long`, and pointers are all 32 bits; LP64 means `long` and pointers are 64 bits; LLP64 means only `long long` and pointers are [14]. Table III shows the sizes Clang uses for several targets.

The table holds a lesson that the names alone hide: the same processor runs under different data models. On x86-64 and AArch64, `long` is eight bytes under Linux but four under Windows [15]. The data model is therefore not a property of the hardware at all, but a choice made by the operating system and its ABI, and a C program that assumes `long` can hold a pointer is correct on one and wrong on the other.

Key point. Categories are names for real differences. CISC and RISC describe where complexity lives; L1, L2, and L3 describe distance; ILP32, LP64, and LLP64 describe type widths chosen by the platform. Understanding the difference makes the name unnecessary to memorize.

IV. The Minimal Machine

If hardware differs so much, on what does it agree? The answer is a small set of operations that every general-purpose processor can perform, each directly or, as Table II showed, through a short sequence of the others.

A. Why Not a Single Instruction?

The set could in principle be made extremely small. It is a known result that a machine with a single instruction, for example "subtract and branch if the result is negative", can compute anything an ordinary computer can [16], [17]. Such machines are mathematically elegant and practically useless, because each instruction does several things at once and simple tasks need long sequences. We therefore seek a *practical* minimum: a set in which each operation does one job, and no job requires another operation to be misused.

B. The Operations

Table IV lists the operations of such a minimal machine. They fall into four groups.

TABLE IV
A Practical Minimal Machine

Operation	Meaning
LOAD, STORE	Copy a value between memory and a register
MOV	Copy a register, or place a constant in one
ADD, SUB	Integer addition and subtraction
AND, OR, XOR	Bitwise logic
SHL, SHR	Shift bits left or right
CMP	Compare two values and record the result
BR	Branch if the last comparison holds
JMP	Jump unconditionally
CALL, RET	Enter a subroutine and return from it

TABLE V
C Constructs as Minimal-Machine Patterns

C construct	Minimal-machine pattern
`x = y + z`	LOAD y, LOAD z, ADD, STORE x
`a[i]`	SHL i by element size, ADD base, LOAD
`*p`	LOAD from the address in p
`s.f`	ADD constant offset of f, then LOAD or STORE
`if (c) A else B`	CMP, BR to B, A, JMP past B
`while (c) A`	CMP, BR out, A, JMP back
`f(x)`	place argument, CALL; in f, ..., RET
`a * b`	hardware multiply if present, else a routine of ADD, SHL, BR

Data movement (LOAD, STORE, MOV) is needed because computation happens in registers but data lives in memory. *Arithmetic and logic* (ADD, SUB, AND, OR, XOR, SHL, SHR) transform values; multiplication and division are deliberately absent, because they can be built from these, as RV32I and the Cortex-M0 show. *Control* (CMP, BR, JMP) chooses what happens next; Böhm and Jacopini showed that sequence, selection, and repetition suffice to express any computation [18], and these three operations provide exactly selection and repetition. *Subroutines* (CALL, RET) let code be reused, with the memory operations providing the stack on which arguments and return addresses are kept.

Every processor targeted in this paper implements each of these operations, though with different encodings, and sometimes combined into larger instructions: x86-64 can fold a LOAD, ADD, and STORE into one instruction (Table I), and RISC-V fuses CMP and BR into single compare-and-branch instructions such as `blt`, "branch if less than". Operations outside this set, such as multiplication, division, floating point, and vector instructions, may or may not be present, and a portable language cannot assume them.

Key point. Beneath the diversity lies agreement: roughly a dozen practical operations that every general-purpose machine can perform. Everything else is optional.

V. C as Portable Assembly

C turned out to fit this situation remarkably well. It lets a programmer write in terms close to the minimal machine while leaving the choice of actual instructions to the compiler.

A. Origins

C grew out of BCPL, designed by Martin Richards, and B, written by Ken Thompson for the PDP-7 minicomputer. Dennis Ritchie developed C at Bell Labs in the early 1970s, adding types so that it could take advantage of the byte-addressed PDP-11, and used it to rewrite the Unix operating system [19]. The language was described for a wide audience by Kernighan and Ritchie in 1978 [20], [21]. Portability was not its first aim: Ritchie wrote that "C was not originally designed with portability as a prime goal". It became portable because it stayed close to what machines have in common, so close that it is now widely used as a compilation target for other languages, "essentially, as a portable assembly language" [19].

A small historical point illustrates how easily categories become myths. It is often said that C's `++` and `--` operators were invented to exploit the PDP-11's automatic increment hardware. Ritchie himself wrote that this is "historically impossible": Thompson invented the operators for B, before the PDP-11 existed [19].

B. Constructs as Patterns

Each C construct can be read as a pattern of minimal-machine operations. Table V gives the correspondence. An array access, for example, is not a separate concept: the C standard defines `a[i]` to mean exactly `*(a + i)` [22], which is an address computation (a shift or multiply of `i` by the element size and an ADD) followed by a LOAD. A `for` loop is a CMP and BR at the top, the body, an ADD to the counter, and a JMP back.

This explains the observation about increments that motivates many discussions of efficiency. When `i = i + 1`, `++i`, and `i++` are used as statements on their own, they mean the same thing, and the compiler produces the same code for all three. With GCC on x86-64 they compile to identical instructions even without optimization, and at `-O2` each becomes a single instruction. The spellings differ in cost only when they differ in meaning: `old = i++` requires the old value to be kept as well as the new one, so it needs more work than `++i`.

C. The Abstract Machine and the As-If Rule

The C standard does not describe any real processor. It defines the meaning of a program in terms of an *abstract machine* [22], [23], and permits a compiler to produce any instructions whose observable behavior matches what that abstract machine would do. This permission, often called the *as-if rule*, is what makes the three increments equivalent, and what allowed Clang to turn the simple summing loop of Section VIII into vector instructions on x86-64 and AArch64 while leaving it as a plain loop on RV32I. The abstract machine is the contract; the instructions are the compiler's business.

D. The Gaps

How can one contract hold on machines that disagree? C's answer is to define behavior precisely where hardware agrees and to leave it open where it does not. The standard distinguishes, among others, three kinds of openness [22]:

- *Implementation-defined* behavior is chosen and documented by each implementation, for example whether plain `char` is signed.
- *Unspecified* behavior allows several outcomes without requiring the choice to be documented, for example the order in which function arguments are evaluated.
- *Undefined* behavior imposes no requirements at all, for example overflow of a signed integer, shifting a value by at least its width, or modifying a variable twice without an intervening sequence point.

These gaps are not defects. Each corresponds to a place where machines genuinely differ: in the width of a register, in how arithmetic overflows, in what a shift instruction does with a large count. By leaving the gaps open, C allows each implementation to use the machine's natural instruction without adding checks, which is the essence of being portable assembly.

Key point. C is portable because it is precise where hardware agrees and silent where it does not. The gaps are the price of mapping directly onto every machine.

VI. Where Values Live: Manual and Automatic Memory

The minimal machine of Table IV can LOAD from an address and STORE to one, and nothing more. It has no instruction that creates an object, reserves memory for it, or releases that memory when the object is no longer needed. The *lifetime* of a value, the period during which a region of memory belongs to it, is therefore not a hardware concept at all. It is a convention, built from ordinary operations, and every language must decide who keeps that convention: the language itself, a runtime system, or the programmer.

A. Three Storage Durations

C makes the choice explicit. The standard states that "there are three storage durations: static, automatic, and allocated" [22], and each maps onto the minimal machine in a different way. The following function bodies illustrate them:

```
static int counter;               /* static    */
void f_static(void) { counter += 1; }
void f_auto(void)   { int buf[16]; use(buf); }
void f_heap(void)   { int *p = malloc(64);
                      use(p); free(p); }
```

Compiled by Clang for x86-64, they become:

```
f_static: incl  counter(%rip)       # fixed address
f_auto:   subq  $72, %rsp           # reserve stack
          movq  %rsp, %rdi
          callq use
          addq  $72, %rsp           # release stack
f_heap:   movl  $64, %edi
          callq malloc              # library call
          ...
          jmp   free                # library call
```

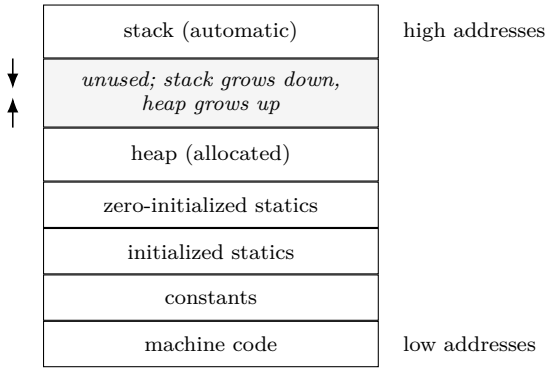


Fig. 2. A typical address-space layout of a running C program. The arrangement is set by the operating system and ABI, not by the C standard.

Static storage exists for the whole run of the program. Its address is fixed when the program is linked, so using it costs nothing beyond the access itself: the increment above is a single instruction naming a fixed location. *Automatic* storage belongs to a block of code and lives on the stack. Reserving it is one SUB from the stack pointer, and releasing it is one ADD, performed automatically when the function returns. *Allocated* storage is obtained by calling `malloc` and released by calling `free` [22]. These are not operators of the language but ordinary library functions, built from the minimal operations, which keep their own records of which regions of a large block of memory are in use [24].

B. The Layout of a Running Program

On a typical system the three durations occupy separate regions of a program's address space, as Fig. 2 shows. The machine code sits in one region, constants and static variables in others, with zero-initialized statics kept apart so that they occupy no space in the program file. Above them lies the *heap*, from which `malloc` draws memory, and near the top of the address space the *stack*, which on all the targets in this paper grows toward lower addresses as functions are called [25], [26], [27]. None of this arrangement is part of the C standard, which speaks only of storage durations. The layout is a convention of the operating system and the ABI, which is why the same program places its data at different addresses on different systems, and often on different runs.

C. Automatic Memory in C, and Beyond It

Two of the three durations are therefore managed without the programmer's involvement. Static storage needs no management, and automatic storage is reclaimed by the function's return, which is why C calls it "automatic". Its limitation is that its lifetime is tied to the block that created it: a function cannot return the address of one of its own local variables and expect it to remain valid.

Many other languages automate allocated storage as well. LISP introduced the automatic reclamation of storage that can no longer be reached, soon known as *garbage collection*, in which a runtime system periodically finds such objects and reclaims them [28]. *Reference counting* keeps, with each object, a count of the references to it, and frees the object when the count reaches zero [29]. Rust takes a third approach, checking at compile time that each value has a single owner and releasing the value when its owner goes out of scope [30]. Each of these automates lifetime at a cost: a collector that runs at times the program does not choose, counters that must be updated on every copy, or language rules that restrict how programs are written.

C deliberately provides none of these, and the reason follows from the argument of Section V. The hardware has no notion of lifetime, and a portable assembly language does not add hidden machinery that the hardware lacks. In C, every cost of memory management is visible as code the programmer wrote or a function the programmer called.

D. The Price of Manual Memory

The price of that visibility is responsibility. Allocated storage stays allocated until it is freed, so forgetting to free it causes a *leak*. Freeing it twice, or freeing memory that did not come from `malloc`, is undefined behavior [22]. Using memory after it has been freed is undefined too, because the standard declares the value of a pointer indeterminate once the object it points to has reached the end of its lifetime [22]. And `malloc` may fail, returning a null pointer that must be checked. None of these mistakes is detected by the compiler, and many produce no visible symptom until much later. They are the memory equivalent of the gaps of Section V: places where the language promises nothing, and correctness rests entirely on the program.

Key point. The hardware knows addresses, not lifetimes. C offers static and automatic storage that manage themselves and allocated storage that the programmer must manage by hand, keeping every cost visible.

VII. Where the Abstraction Meets the Machine

The abstract machine of Section V is a contract about a program's observable behavior, and the as-if rule lets the compiler choose any instructions that honor it. Three situations strain that arrangement, because in each the program must deal with something outside the abstract machine: a hardware device, separately compiled code, or another processor running at the same time.

A. Devices and `volatile`

The most common reason to want portable assembly is to control hardware directly. On most systems a device is controlled through *memory-mapped input/output*: its registers appear at fixed addresses, so a LOAD reads the device's status and a STORE issues a command. To the minimal machine these are ordinary memory operations. To the compiler, however, they look like accesses that can be optimized, and the as-if rule permits it. Consider a function that waits for a device to become ready, and one that sends two commands:

```
void wait(uint32_t *status)
  { while ((*status & 1u) == 0) { } }
void cmd(uint32_t *reg) { *reg = 1; *reg = 2; }
```

Clang compiles `wait` on x86-64 to read the status *once*; if the device is not yet ready, the function then spins forever without reading it again, because nothing in the abstract machine could change the value. It compiles `cmd` to a single store of 2, deleting the first command, because the first store is immediately overwritten. Both translations are correct under the as-if rule, and both are wrong for a device.

The `volatile` qualifier exists for exactly this case. An object with volatile-qualified type "may be modified in ways unknown to the implementation or have other unknown side effects" [22], so every access to it in the program must be performed, in order, as an access to memory. With the pointers declared `volatile uint32_t *`, Clang reads the status on every iteration and performs both stores. `volatile` is the one place where C lets the programmer say that a memory operation is itself the point, not merely a means of computing a value.

Two limits matter. First, the address of a device register, written as an integer converted to a pointer, is implementation-defined [22], so code that names device addresses is inherently tied to one platform; the canonical form isolates it in one small module. Second, `volatile` controls only what the compiler emits. It does not make an operation atomic, and it does not order memory accesses between processors (Section VII-C).

B. Calling Conventions

The minimal machine has CALL and RET, and every target implements them. What the C standard does not say is *where* a function finds its arguments and leaves its result. That is fixed by each platform's *application binary interface* (ABI), the binary-level contract that lets code compiled at different times, by different compilers, call each other correctly. The same function compiled for three 64-bit targets shows the difference:

```
long add3(long a, long b, long c)
  { return a + b + c; }
struct big { long x, y, z; };
struct big mk(long v)
  { struct big r = { v, v, v }; return r; }
```

The three arguments of `add3` arrive in registers `rdi`, `rsi`, and `rdx` on x86-64, in `x0`–`x2` on AArch64, and in `a0`–`a2` on RISC-V. The 24-byte structure returned by `mk` is too large for registers on all three, so the caller supplies a hidden address at which to store it: passed in `rdi` on x86-64, in the dedicated result register `x8` on AArch64, and in `a0` on RISC-V [25], [26], [27]. The ABI is the C standard's counterpart one layer down. The standard makes source code portable across machines; an ABI makes compiled

code compatible on one machine. A program that relies on a particular calling convention, for example by calling a function through a pointer of the wrong type, steps outside both.

C. Concurrency

The largest gap of all concerns programs that run on several processors at once. Modern multicore processors diverged sharply in how they order memory operations as seen by other processors. x86 processors follow *total store order*, in which the only reordering visible to other processors is a later load completing before an earlier store that is still buffered [31]. RISC-V's memory model permits considerably more reordering unless the program inserts explicit ordering instructions [3], and the listings below show that AArch64 likewise requires explicit ordering where x86 does not. C99 says nothing about this: it has no threads, no atomic operations, and no rules for memory shared between them. All of that was left to platform libraries such as POSIX threads [32]. Boehm showed that this arrangement is fundamentally unsound, because a compiler that knows nothing of threads may legally transform code in ways that break correct threaded programs [33]. The C11 standard therefore added a memory model and atomic operations [34], building on work for C++ [35].

The divergence shows clearly in what one C11 statement becomes. A sequentially consistent atomic store, `atomic_store(&flag, 1)`, compiles on x86-64 to `xchg`, an instruction that also forces ordering, and on AArch64 to `stlr`, a store with release ordering. A weaker release store compiles to a plain `mov` on x86-64, whose ordering is already strong enough, but still to `stlr` on AArch64. One portable statement, chosen correctly, produces whatever each machine needs; a C99 program, lacking such statements, must rely on a platform library to do the same.

Key point. Devices, binary interfaces, and multiple processors are where the abstract machine meets the real one. C provides `volatile` for devices, leaves calling conventions to the ABI, and in C99 leaves concurrency entirely to the platform.

VIII. Abstraction and Canonical Forms

The bargain has a consequence that is easy to overlook. A C program means the same thing on every machine only if it stays out of the gaps. Code that steps into them still compiles, but its translation is no longer fixed by the language: it depends on the target, the compiler, or the optimization level. For each construct, therefore, there is a *canonical form*, a way of writing it that keeps the program inside the region where every machine agrees. The canonical form is what makes the abstraction correct.

A. The Generating Principle

The forms that follow should not be read as another list of rules to memorize. Each is derived from a single principle with three parts. A canonical form must

1) *stay inside defined behavior*, using only what the standard defines identically for every implementation, or making any implementation-defined choice explicit in the code;
2) *have a predictable translation*, so that its meaning, and roughly its cost, can be worked out from the minimal-machine patterns of Table V without knowing the target; and
3) *make departures detectable*, so that the compiler, its warnings, or a static analyzer rejects code written another way.

Given a construct, one finds its canonical form by asking where it touches a gap of Section V and choosing the spelling that avoids the gap, translates predictably, and can be checked. Table VI at the end of this section summarizes the result for every construct discussed.

B. Integers

The width of `int` and the signedness of `char` vary between implementations. Consider a function that tests whether a character is negative:

```
int is_neg(char c) { return c < 0; }
```

On x86-64, where `char` is signed [25], Clang tests the sign bit. On AArch64 and RISC-V, where plain `char` is unsigned [26], [27], it compiles the function to "return 0":

```
# x86-64                    # AArch64
shrb   $7, %dil             mov  w0, wzr
movzbl %dil, %eax           ret
retq
```

Both translations are correct; the program was not. Signed overflow is subtler still. Because it is undefined, the compiler may assume it never happens:

```
int s(int x)      { return x + 1 > x; }
int u(unsigned x) { return x + 1 > x; }
```

At `-O2`, Clang compiles `s` to "return 1", reasoning that `x + 1` cannot overflow, but keeps a real comparison in `u`, because unsigned arithmetic is defined to wrap around. Compilers exploit such assumptions routinely, and code that relies on overflow behaving like the hardware can silently change meaning [36], [37]. A third trap lies in C's conversion rules. Arithmetic is never performed on types narrower than `int`: in the *integer promotions*, a smaller value "is converted to an int" whenever `int` can represent all its values [22]. So for two `uint8_t` variables holding 200 and 100, `a + b > 255` is true, because the sum is computed as an `int`. The promotion can also introduce overflow where none seems possible: multiplying two `uint16_t` values of 65535 multiplies two `int`s, which overflows a 32-bit `int`, and is therefore undefined; a sanitizer reports "signed integer overflow: 65535 * 65535 cannot be represented in type 'int' ". In the other direction, the *usual arithmetic conversions* convert a signed operand to unsigned when it meets an unsigned operand of the same rank [22], so `-1 < 1u` is false; GCC warns about the comparison only when asked to. These rules follow from

the minimal machine, whose arithmetic works on whole registers, but they must be known rather than guessed.

The canonical form is to use the fixed-width types of `<stdint.h>`, to cast operands explicitly to the type in which arithmetic should be performed (for example, multiplying `(uint32_t)x * x`), to keep signed and unsigned values out of the same comparison, to use `signed char` or `unsigned char` explicitly when the sign matters, to use unsigned types for bit manipulation and wrap-around arithmetic, and to check ranges before an operation rather than after it.

C. Expressions

An expression should change at most one object, and should not read an object it changes elsewhere in the same expression. The statement `i = i++ + 1` modifies `i` twice; its behavior is undefined, and GCC warns that "operation on 'i' may be undefined". The canonical form gives each side effect its own statement. Within that rule, `++i`, `i++`, and `i += 1` are interchangeable as statements, and a project may simply choose one spelling for consistency.

D. Loops

A loop translates predictably when the number of iterations can be determined before it starts. The canonical counted loop is:

```
for (size_t i = 0; i < n; i++) { /* body */ }
```

with `n` not changed by the body, `i` not modified inside it, and an unsigned index type wide enough to address any object. In this form the compiler can compute the trip count, which is what allowed Clang to vectorize the summing loop on x86-64 and AArch64, and a verification tool can prove that the loop terminates. Safety-critical coding rules make the same demand: Holzmann's "Power of Ten" requires every loop to have a fixed upper bound that a checking tool can establish [38]. Loops whose exit depends on data that may never arrive, or whose counter is adjusted inside the body, are harder for both compilers and people to reason about.

E. Arrays and Pointers

Because `a[i]` is defined as `*(a + i)`, every array access is pointer arithmetic, and pointer arithmetic is defined only within an array, plus the position one past its last element [22]. An access outside that range is undefined, not merely "a read of whatever is there". The standard also restricts which types may be used to access an object, the *aliasing rule*, so that compilers can keep values in registers without reloading them after every store [22]. The canonical form keeps every index within bounds, passes the length with every array, accesses objects through their declared type, and uses `restrict` to promise that two pointers do not overlap when that is true.

F. Structures

A structure is a list of fields at fixed offsets, and the compiler may insert padding between fields to satisfy alignment [22]. On x86-64, a structure holding a 1-byte field, a 4-byte field, and another 1-byte field, in that order, occupies 12 bytes; reordering the fields from largest to smallest reduces it to 8. Layout therefore depends on both field order and target alignment rules. The canonical form orders fields by decreasing size, never assumes a layout that has not been checked, and verifies important sizes at compile time. In C99, which lacks a static assertion keyword, the classic technique declares an array whose size becomes negative when the condition fails:

```
#define STATIC_ASSERT(c, name) \
  typedef char name[(c) ? 1 : -1]
STATIC_ASSERT(sizeof(struct pkt) == 8, pkt_size);
```

G. Unions

A union stores several possible fields in the same memory. Writing one field and reading another reinterprets the bytes, and the result depends on the target's representation of each type, including its byte order. The canonical form is the *tagged union*: a structure containing a tag that records which field is valid, together with the union itself, with every access checking the tag first. This turns an unchecked reinterpretation into an explicit choice.

H. Selection: `switch`

A `switch` statement [22] illustrates the as-if rule at work on control flow. When the cases are dense, Clang on x86-64 compiles it to a bounds check followed by a single indexed LOAD from a table; when they are sparse, it compiles the same construct to a chain of CMP and BR instructions. The programmer writes one construct and the compiler chooses the translation. The canonical form serves that choice: every `switch` has a `default` label, every case ends in `break` or `return` unless falling through is explicitly marked, and the controlling expression has an integer type whose range of values is known, so that every possible value is accounted for.

I. Functions

Every function should be declared with a full prototype before use, so that the compiler converts and checks arguments; C99 already removed the earlier rule that an undeclared function implicitly returns `int` [22]. Functions with a fixed number of arguments are easier to check than variadic ones, and avoiding recursion keeps the stack depth known, another of the Power of Ten rules [38].

J. Bytes and Byte Order

Whenever data leaves the program as bytes, in a file or a network packet, byte order and alignment become visible. A tempting way to read a 32-bit big-endian value is to convert the byte pointer to a `uint32_t *` and load

through it, but that steps into three gaps at once: the result depends on the machine's byte order, the address may not be suitably aligned, and accessing bytes through an unrelated type breaks the aliasing rule [22]. The canonical form assembles the value from individual bytes using only minimal-machine operations:

```
uint32_t load_be(const uint8_t *b) {
  return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16)
       | ((uint32_t)b[2] <<  8) |  (uint32_t)b[3];
}
```

This means the same thing on every machine, and it costs little. Clang recognizes the pattern and compiles it on x86-64 to a single load followed by `bswap`, and on AArch64 to a load followed by `rev`; on RV32I, which has no byte-reversal instruction, it produces four byte loads with shifts and ORs. The canonical form expresses intent, and each compiler chooses the best instructions available. Note also that exact-width types such as `uint32_t` are themselves optional in C99 [22]. They exist on every mainstream platform, but a program that requires them assumes types without padding bits, and one that uses `uint8_t`, as `load_be` does, assumes 8-bit bytes.

K. Hardware Access

Every access to a device register goes through a `volatile`-qualified pointer, so that the compiler performs it exactly as written (Section VII). Device addresses are implementation-defined, so they belong in one small, clearly marked module rather than throughout the program. Each statement should perform a single volatile access, because the order of two volatile accesses within one expression is not always specified. And `volatile` is never used as a substitute for synchronization between threads.

L. Shared Data Between Threads

Because C99 has no memory model, a C99 program that shares data between threads must do so only through the synchronization operations of its platform, such as the mutexes of POSIX threads [32], and must not rely on ordinary or `volatile` variables for communication, since neither the compiler nor the processor is obliged to preserve the intended order [33]. Where C11 is available, the canonical form is an atomic object accessed with an explicit memory order, which, as Section VII-C showed, compiles to the right instructions for each machine.

M. Memory and Lifetime

Because the language does not track lifetimes, the canonical forms for memory make lifetimes simple enough to see. Prefer static and automatic storage, whose lifetimes the compiler manages, and give arrays sizes known at compile time; C99's variable-length arrays place a size chosen at run time on the stack, where an unexpectedly large value can exhaust it. When allocated storage is needed, allocate it during initialization and keep it for the life of the program, as the Power of Ten rules require: "do not use dynamic memory allocation after initialization" [38]. When many objects share a lifetime, allocate them from a single region, often called an *arena*, and release the whole region at once [39]; one release replaces many individual calls to `free`, and none can be forgotten. Give every allocation a single owner responsible for freeing it, check every result of `malloc`, and set a pointer to null once its memory has been freed, so that a later mistaken use fails predictably rather than silently.

N. Platform-Specific Code

Some code cannot be portable: a device driver, a routine that uses a processor's vector instructions, a workaround for one compiler. The canonical response is not to avoid such code but to confine it. Platform-specific parts are placed in their own small modules behind a portable interface, and the build selects the right module, or conditional compilation selects the right variant [22]:

```
#if defined(__x86_64__)
#  include "port_x86_64.h"
#elif defined(__aarch64__)
#  include "port_aarch64.h"
#else
#  include "port_generic.h"   /* portable C only */
#endif
```

The generic variant, written only in canonical forms, guarantees that the program runs correctly everywhere; the specific variants exist only to run faster or to reach hardware that portable C cannot. Conditional compilation scattered through the program, by contrast, multiplies the number of distinct programs that must each be correct.

O. Making the Wrong Form Fail

A canonical form is most useful when departing from it is an error rather than a style issue. Some of this can be achieved within C99 itself. Wrapping a value in a one-member structure creates a distinct type, so that, for example, a `meters_t` cannot be passed where a `seconds_t` is expected; GCC rejects the mistake with "incompatible type for argument". Compile-time assertions turn layout assumptions into checked facts. Beyond the language, strict compiler settings (`-std=c99 -Wall -Wextra -Werror`, together with conversion and sign-comparison warnings) reject many departures, and published rule sets such as MISRA C [40] and the Power of Ten [38], enforced by static analysis, cover more. No single layer catches everything, so each canonical form should state which layer enforces it.

Key point. Canonical forms keep a program inside the region where every machine agrees. Outside it, code still compiles, but its meaning depends on the target. Wherever possible, the wrong form should fail to compile.

IX. Discussion

Three limits of the argument deserve mention. First, canonical forms make *correctness* portable, but not *performance*. Caches, branch predictors, and vector units belong

TABLE VI
SUMMARY OF CANONICAL FORMS: THE GAP EACH AVOIDS AND THE LAYER THAT ENFORCES IT

Construct	Gap avoided	Canonical form	Enforced by
Integers	Width of `int`, sign of `char`, signed overflow, promotions	Fixed-width types; explicit `signed/unsigned char`; unsigned for bits; explicit casts; range checks before operations	`-Wconversion`, `-Wsign-compare`; sanitizers; static analysis
Expressions	Unsequenced modification, unspecified evaluation order	At most one side effect per expression	`-Wsequence-point`; static analysis
Loops	Unknown trip count	Counted loop with invariant bound and unmodified index	Static analysis; coding rules
Arrays, pointers	Out-of-bounds access, aliasing	Length passed with every array; access through declared type; `restrict` where true	Sanitizers; static analysis
Structures	Padding and alignment	Fields ordered by size; sizes checked at compile time	Compile-time assertions
Unions	Reinterpreted representation	Tagged union, tag checked on every access	Review; static analysis
Selection	Unhandled values, accidental fallthrough	`default` present; every case terminated	`-Wswitch`, `-Wimplicit-fallthrough`
Functions	Unchecked arguments, unbounded stack	Full prototypes; fixed arguments; no recursion	`-Werror`; static analysis
Bytes	Byte order, alignment, aliasing	Assemble values with shifts and ORs	Review; static analysis
Hardware access	Accesses removed or merged by the as-if rule	`volatile` pointer; one access per statement; isolated module	Review
Shared data	No memory model in C99	Platform synchronization; C11 atomics where available	Thread sanitizer; review
Memory	Leaks, double free, use after free	Static and automatic storage; allocate at initialization; arenas; single owner	Sanitizers; static analysis
Platform code	Non-portable constructs	Confined to modules behind a portable interface, with a generic fallback	Build structure; review
Types in general	Mixing incompatible values	Distinct struct types for distinct quantities	The compiler itself

to the microarchitecture, which C deliberately does not describe. Chisnall has argued that, for this reason, C is no longer a low-level language: its abstract machine resembles the processors of the 1970s more than modern ones, and compilers must work hard to map it onto them [41]. Two C programs that are equally correct can differ greatly in speed because of how they use memory, as Drepper showed in detail [1]. Hardware and compilers offer hints and controls for this, such as prefetch instructions and processor-specific build options, but by design they do not change a program's meaning.

Second, the canonical forms for memory rely on discipline that C itself cannot check. Languages that track ownership at compile time [30] show that the rule "every allocation has one owner" can be made a compile-time error rather than a convention; within C, the closest substitutes are simple allocation patterns, such as arenas, that leave little to get wrong.

Third, the minimal machine of Section IV is an idealization. Real portable programs also rely on operations outside it, such as multiplication and floating point, and on a C library. The argument still holds, because these are supplied either by hardware or by routines built from the minimal operations, but their cost varies more from machine to machine than the cost of the core operations. Floating point in particular lies outside the scope of this paper. Almost all current hardware implements the IEEE 754 standard [42], but C99 still permits intermediate results to be computed at higher precision than their declared type, as reported by the macro `FLT_EVAL_METHOD` [22]; on older x86 processors this made results depend on the compiler and the optimization level [43]. Floating-point canonical forms deserve a treatment of their own.

Neither limit weakens the main point. The categories that programmers are asked to memorize, from CISC and RISC to L1 and L2 to prefix and postfix increment, are consequences of a small number of facts: hardware diverged, it still agrees on a small core, C was designed to sit on that core, and it remains correct only where programs keep to canonical forms.

X. Conclusion

C earns the description "portable assembly" not by matching any machine but by matching what all machines share. Its abstract machine is precise where hardware agrees and deliberately silent where hardware differs, and its constructs are short, predictable patterns over a dozen practical operations. Where the program must touch the machine directly, through devices, binary interfaces, or shared memory, C supplies a minimal mechanism or defers to the platform. The price of that design is that correctness depends on discipline: each construct must be written in a form that stays inside the agreed region, and the most robust programs make the alternatives impossible to compile. Seen this way, the long lists of categories that surround C and hardware become unnecessary to memorize. They can be derived from the structure beneath them.

References

[1] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., Nov. 2007. [Online]. Available: https://www.akkadia.org/drepper/cpumemory.pdf

[2] J. von Neumann, "First draft of a report on the EDVAC," Moore School of Electrical Engineering, University of Pennsylvania, Philadelphia, PA, USA, Tech. Rep., Jun. 1945, reprinted in IEEE Ann. Hist. Comput., vol. 15, no. 4, pp. 27–75, 1993, doi: 10.1109/85.238389.

[3] *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V Foundation, Dec. 2019, document Version 20191213. [Online]. Available: https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf

[4] ARM Limited, *ARMv6-M Architecture Reference Manual*, ARM Limited, Sep. 2010. [Online]. Available: https://developer.arm.com/documentation/ddi0419/latest/

[5] Free Software Foundation, *GNU Compiler Collection (GCC) Internals, Sec. 3.1: Routines for Integer Arithmetic*, Free Software Foundation, 2026. [Online]. Available: https://gcc.gnu.org/onlinedocs/gccint/Integer-library-routines.html

[6] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 2 (2A, 2B, 2C & 2D): Instruction Set Reference, A–Z*, Intel Corporation, Sep. 2026. [Online]. Available: https://cdrdv2.intel.com/v1/dl/getContent/671110

[7] A. Fog, *Instruction Tables: Lists of Instruction Latencies, Throughputs and Micro-Operation Breakdowns for Intel, AMD and VIA CPUs*, Sep. 2025, optimization manual 4, last updated 2025-09-20. [Online]. Available: https://www.agner.org/optimize/instruction_tables.pdf

[8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019, a 7th edition (with C. Kozyrakis, ISBN 978-0-443-15406-5) appeared in 2025.

[9] G. M. Amdahl, G. A. Blaauw, and J. Frederick P. Brooks, "Architecture of the IBM System/360," *IBM J. Res. Develop.*, vol. 8, no. 2, pp. 87–101, Apr. 1964.

[10] D. A. Patterson and D. R. Ditzel, "The case for the reduced instruction set computer," *ACM SIGARCH Comput. Archit. News*, vol. 8, no. 6, pp. 25–33, Oct. 1980.

[11] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Comput. Archit. News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.

[12] J. S. Liptay, "Structural aspects of the System/360 Model 85, II: The cache," *IBM Syst. J.*, vol. 7, no. 1, pp. 15–21, 1968.

[13] D. Cohen, "On holy wars and a plea for peace," *Computer*, vol. 14, no. 10, pp. 48–54, Oct. 1981.

[14] The Open Group, "64-bit programming models: Why LP64?" The Open Group (Aspen Data Model Committee), White Paper, 1997. [Online]. Available: https://archive.opengroup.org/public/tech/aspen/lp64_wp.htm

[15] Microsoft, "Abstract data models," Microsoft Learn, Win32 Programming Guide for 64-bit Windows, 2026, accessed Nov. 2026. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/winprog64/abstract-data-models

[16] F. Mavaddat and B. Parhami, "URISC: The ultimate reduced instruction set computer," *Int. J. Elect. Eng. Educ.*, vol. 25, no. 4, pp. 327–334, Oct. 1988.

[17] A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem," *Proc. London Math. Soc.*, vol. s2-42, no. 1, pp. 230–265, 1937, issued in parts in Nov.–Dec. 1936; correction in vol. s2-43, pp. 544–546, 1938.

[18] C. Böhm and G. Jacopini, "Flow diagrams, Turing machines and languages with only two formation rules," *Commun. ACM*, vol. 9, no. 5, pp. 366–371, May 1966.

[19] D. M. Ritchie, "The development of the C language," in *Proc. 2nd ACM SIGPLAN Conf. History of Programming Languages (HOPL-II)*. Cambridge, MA, USA: ACM, Apr. 1993, pp. 201–208, also published in ACM SIGPLAN Notices, vol. 28, no. 3, Mar. 1993. [Online]. Available: https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.html

[20] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 1st ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1978.

[21] ——, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[22] ISO/IEC JTC1/SC22/WG14, *ISO/IEC 9899:1999, Programming Languages – C*, International Organization for Standardization, Geneva, Switzerland, Dec. 1999. [Online]. Available: https://www.iso.org/standard/29237.html

[23] ——, "ISO/IEC 9899:TC3, committee draft N1256 (c99 with technical corrigenda 1, 2, and 3)," ISO/IEC JTC1/SC22/WG14, Committee Draft N1256, Sep. 2007. [Online]. Available: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf

[24] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, "Dynamic storage allocation: A survey and critical review," in *Proc. Int. Workshop on Memory Management (IWMM)*, ser. Lecture Notes in Computer Science, H. G. Baker, Ed., vol. 986. Berlin, Heidelberg: Springer, 1995, pp. 1–116.

[25] *System V Application Binary Interface: AMD64 Architecture Processor Supplement (with LP64 and ILP32 Programming Models)*, x86-64 psABI Working Group, 2026, version 1.0 (living document, accessed Nov. 2026). [Online]. Available: https://gitlab.com/x86-psABIs/x86-64-ABI

[26] Arm Limited, *Procedure Call Standard for the Arm 64-bit Architecture (AArch64)*, Arm Limited, 2026. [Online]. Available: https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst

[27] *RISC-V ABIs Specification*, RISC-V International, Nov. 2022, document Version 1.0. [Online]. Available: https://github.com/riscv-non-isa/riscv-elf-psabi-doc/releases/tag/v1.0

[28] J. McCarthy, "Recursive functions of symbolic expressions and their computation by machine, part I," *Communications of the ACM*, vol. 3, no. 4, pp. 184–195, Apr. 1960.

[29] G. E. Collins, "A method for overlapping and erasure of lists," *Communications of the ACM*, vol. 3, no. 12, pp. 655–657, Dec. 1960.

[30] S. Klabnik, C. Nichols, and C. Krycho, "The Rust programming language," Online book, ch. 4, "Understanding Ownership", 2026, accessed: Nov. 5, 2026. [Online]. Available: https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

[31] S. Owens, S. Sarkar, and P. Sewell, "A better x86 memory model: x86-TSO," in *Theorem Proving in Higher Order Logics (TPHOLs)*, ser. Lecture Notes in Computer Science, vol. 5674. Berlin, Heidelberg: Springer, Aug. 2009, pp. 391–407.

[32] IEEE and The Open Group, *IEEE Std 1003.1-2024: Standard for Information Technology—Portable Operating System Interface (POSIX) Base Specifications, Issue 8*, IEEE and The Open Group, 2024, the Open Group Base Specifications Issue 8. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/

[33] H.-J. Boehm, "Threads cannot be implemented as a library," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*. Chicago, IL, USA: ACM, Jun. 2005, pp. 261–268.

[34] ISO/IEC JTC1/SC22/WG14, *ISO/IEC 9899:2011, Information Technology – Programming Languages – C*, 3rd ed., International Organization for Standardization, Geneva, Switzerland, Dec. 2011, withdrawn; revised by ISO/IEC 9899:2018. [Online]. Available: https://www.iso.org/standard/57853.html

[35] H.-J. Boehm and S. V. Adve, "Foundations of the C++ concurrency memory model," in *Proc. 29th ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*. Tucson, AZ, USA: ACM, Jun. 2008, pp. 68–78.

[36] C. Lattner, "What every C programmer should know about undefined behavior #1/3," LLVM Project Blog, May 2011, part 1 of a 3-part series, posted May 13, 2011. [Online]. Available: https://blog.llvm.org/2011/05/what-every-c-programmer-should-know.html

[37] X. Wang, H. Chen, A. Cheung, Z. Jia, N. Zeldovich, and M. F. Kaashoek, "Undefined behavior: What happened to my code?" in *Proc. Asia-Pacific Workshop on Systems (APSys)*. Seoul, South Korea: ACM, Jul. 2012, pp. 1–7.

[38] G. J. Holzmann, "The power of 10: Rules for developing safety-critical code," *Computer*, vol. 39, no. 6, pp. 95–97, Jun. 2006.

[39] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, Jan. 1990.

[40] MISRA, *MISRA C:2023 – Guidelines for the Use of the C Language in Critical Systems*, The MISRA Consortium Limited, Norwich, U.K., 2023, superseded by MISRA C:2025 (Mar. 2025).

[41] D. Chisnall, "C is not a low-level language," *ACM Queue*, vol. 16, no. 2, pp. 18–30, Apr. 2018, reprinted in Commun. ACM, vol. 61, no. 7, pp. 44–48, Jul. 2018, doi: 10.1145/3209212.

[42] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE, Jul. 2019, iEEE Std 754-2019 (Revision of IEEE Std 754-2008).

[43] D. Monniaux, "The pitfalls of verifying floating-point computations," *ACM Transactions on Programming Languages and Systems*, vol. 30, no. 3, pp. 12:1–12:41, May 2008.